

```

root@server1:~/Desktop
[root@server1 Desktop]# ps lax
F  UID  PID  PPID  PRI  NI   VSZ  RSS  WCHAN  STAT  TTY      TIME  COMMAND
1  0      1      0    20   0 52840 6608 ep_poll Ss    ?      0:18 /usr/lib/systemd/systemd
1  0      2      0    20   0   0     0 kthre  S      ?      0:00 [kthreadd]
1  0      3      2    20   0   0     0 ksoft  S      ?      0:00 [ksoftirqd/0]
1  0      5      2    20   0   0     0 kwork  S<    ?      0:00 [kworker/0:0H]
1  0      6      2    20   0   0     0 kwork  S<    ?      0:00 [kworker/u4:0]
1  0      7      2   -100  -   0     0 migra  S      ?      0:01 [migration/0]
1  0      8      2    20   0   0     0 rcu_gp S      ?      0:00 [rcu_gp]
1  0      9      2    20   0   0     0 rcu_no S      ?      0:00 [rcuob/0]
1  0     10      2    20   0   0     0 rcu_no S      ?      0:00 [rcuob/1]
1  0     11      2    20   0   0     0 rcu_gp S      ?      0:05 [rcu_sched]
1  0     12      2    20   0   0     0 rcu_no S      ?      0:01 [rcuos/0]
1  0     13      2    20   0   0     0 rcu_no S      ?      0:01 [rcuos/1]
1  0     14      2   -100  -   0     0 watch  S      ?      0:00 [watchdog/0]
1  0     15      2   -100  -   0     0 watch  S      ?      0:00 [watchdog/1]
1  0     16      2   -100  -   0     0 migra  S      ?      0:01 [migration/1]
1  0     17      2    20   0   0     0 ksoft  S      ?      0:00 [ksoftirqd/1]
1  0     19      2    20   0   0     0 kwork  S<    ?      0:00 [kworker/1:0H]
1  0     20      2    20   0   0     0 khelp  S      ?      0:00 [khelper]
1  0     21      2    20   0   0     0 kdevt  S      ?      0:00 [kdevtmpfs]
1  0     22      2    20   0   0     0 netns  S<    ?      0:00 [netns]
1  0     23      2    20   0   0     0 writ  S<    ?      0:00 [writeback]
1  0     24      2    20   0   0     0 kinteg S<    ?      0:00 [kintegrityd]
1  0     25      2    20   0   0     0 bios  S<    ?      0:00 [bioset]
1  0     26      2    20   0   0     0 kblock S<    ?      0:00 [kblockd]
1  0     27      2    20   0   0     0 khubd  S      ?      0:00 [khubd]
1  0     28      2    20   0   0     0 md     S<    ?      0:00 [md]
1  0     32      2    20   0   0     0 kswa  S      ?      0:00 [kswapd0]
1  0     33      2    25   5   0     0 ksm_sc Ss    ?      0:00 [ksm]

```

Figure 2: The output of `ps lax`

```

root@server1:~/Desktop
[root@server1 Desktop]# ps -ef
UID  PID  PPID  C  STIME  TTY      TIME  CMD
root  1      0  0  13:38  ?      00:00:18 /usr/lib/systemd/systemd --switched-root --
root  2      0  0  13:38  ?      00:00:00 [kthreadd]
root  3      2  0  13:38  ?      00:00:00 [ksoftirqd/0]
root  5      2  0  13:38  ?      00:00:00 [kworker/0:0H]
root  6      2  0  13:38  ?      00:00:00 [kworker/u4:0]
root  7      2  0  13:38  ?      00:00:01 [migration/0]
root  8      2  0  13:38  ?      00:00:00 [rcu_bh]
root  9      2  0  13:38  ?      00:00:00 [rcuob/0]
root 10      2  0  13:38  ?      00:00:00 [rcuob/1]
root 11      2  0  13:38  ?      00:00:05 [rcu_sched]
root 12      2  0  13:38  ?      00:00:01 [rcuos/0]
root 13      2  0  13:38  ?      00:00:01 [rcuos/1]
root 14      2  0  13:38  ?      00:00:00 [watchdog/0]
root 15      2  0  13:38  ?      00:00:00 [watchdog/1]
root 16      2  0  13:38  ?      00:00:01 [migration/1]
root 17      2  0  13:38  ?      00:00:00 [ksoftirqd/1]
root 19      2  0  13:38  ?      00:00:00 [kworker/1:0H]
root 20      2  0  13:38  ?      00:00:00 [khelper]
root 21      2  0  13:38  ?      00:00:00 [kdevtmpfs]
root 22      2  0  13:38  ?      00:00:00 [netns]
root 23      2  0  13:38  ?      00:00:00 [writeback]
root 24      2  0  13:38  ?      00:00:00 [kintegrityd]
root 25      2  0  13:38  ?      00:00:00 [bioset]
root 26      2  0  13:38  ?      00:00:00 [kblockd]
root 27      2  0  13:38  ?      00:00:00 [khubd]
root 28      2  0  13:38  ?      00:00:00 [md]
root 32      2  0  13:38  ?      00:00:00 [kswapd0]
root 33      2  0  13:38  ?      00:00:00 [ksm]

```

Figure 3: The output of `ps -ef`

- `ps` can be displayed in tree format to view parent/child relationships.
- The default output is unsorted. The display order matches that of the system process table, which reuses table rows as processes die and new ones are created. The output may appear chronological, but this is not guaranteed unless explicitly stated `-O, --sort` options are used.

Controlling jobs

Jobs and sessions: Job control is a command shell feature allowing a single shell instance to run and manage multiple commands. Without job control, a parent shell forks a child process to run a command, sleeping until the child process exits. When the shell prompt redisplay, the parent shell returns. With job control, commands can be selectively suspended, resumed and run asynchronously, allowing the shell to return for additional commands while the child process can run.

A foreground process is a command running in a terminal window. The terminal device ID (tty) is the process's

controlling terminal. Foreground processes receive keyboard-generated input and signals, and are allowed to read from or write to the terminal (e.g., via `stdin` and `stdout`).

A process session is created when a terminal or console first opens (e.g., at login or by invoking a new terminal instance). All processes (e.g., the first command shell, its children and pipelines) initiated from that terminal share the same session ID. Within that session, only one process can be in the foreground at a time.

A background process is started without a controlling terminal because it has no need for terminal interaction. In a `ps` listing, such processes (e.g., services, daemons and kernel process threads) display a question mark (?) in the TTY column, while background processes that (improperly) attempt to read from or write to the terminal may be suspended.

Running jobs in the background

Any command can be started in the background by appending an ampersand (&) to the command line. The bash shell displays a job number (unique to the session) and the PID of the new child process. The command shell does not wait for the child process and redisplay the shell prompt.

The bash command shell tracks jobs, per session, in a table displayed with the `jobs` command as shown in Figure 4.

Background jobs can reconnect to the controlling terminal by being brought to the foreground using the `fg` command with the job ID (%*job number*).

The example `sleep` command is now running on the controlling terminal. The command shell is again asleep, waiting for this child process to exit. To resend it to the background, or to send any command in which the trailing ampersand was not originally included, send the keyboard generated suspend request (`Ctrl-z`) to the process. The suspend request takes effect immediately. The job is placed in the background. The pending output and keyboard typeahead are discarded.

```

root@server1:~/Desktop
[root@server1 Desktop]# sleep 1000 &
[1] 3085
[root@server1 Desktop]# jobs
[1]+  Running                  sleep 1000 &
[root@server1 Desktop]# fg %1
sleep 1000

```

Figure 4: Process is sent to the background and then runs on the control terminal

```

root@server1:~/Desktop
[root@server1 Desktop]# ps j
PPID  PID  PGID  SID  TTY      TPGID  STAT  UID   TIME  COMMAND
638   653   653   653  tty1     653    Ss+   0     0:26 /usr/bin/Xorg :0 -ba
1    1316  1316  1316  tty50    1316   Ss+   0     0:00 /sbin/agetty --keep-
3050  3054  3054  3054  pts/0    3129   Ss    0     0:00 /bin/bash
3054  3129  3129  3054  pts/0    3129   R+    0     0:00 ps j

```

Figure 5: The job display output